DOCUMENTATION TECHNIQUE

Montaza ERP

Laravel 12 | PHP 8.3+ | PostgreSQL | Livewire 3 | Tailwind CSS

Architecture · Modèles de données · Modules fonctionnels · Sécurité · Déploiement

Avril 2026 — Version 1.x

Documentation Technique — Montaza ERP

Version : Laravel 12

PHP 8.3+	***PostgreSQL***	***Livewire 3***

Tailwind CSS

Date de mise à jour : Avril 2026

Table des matières

1. Vue d'ensemble
2. Architecture technique
3. Prérequis et installation
4. Structure du projet
5. Base de données
6. Modules fonctionnels

- 6.1 Gestion commerciale — Commandes (CDE)
- 6.2 Demandes de Prix (DDP)
- 6.3 Affaires
- 6.4 Devis Tuyauterie
- 6.5 Dossiers Devis
- 6.6 Stock & Matières
- 6.7 Tiers — Sociétés & Contacts
- 6.8 Personnel & Congés
- 6.9 Atelier / SAV — Matériel & Réparations
- 6.10 Facturation
- 6.11 Administration

1. Couche modèles (Eloquent)
2. Contrôleurs
3. Services métier
4. Middleware
5. Composants Livewire
6. Helpers globaux
7. Système de notifications
8. Système de permissions & rôles
9. Gestion des médias

10. Historique des modifications (Audit)
11. Routage
12. Frontend
13. Sécurité
14. Tests
15. Déploiement
16. Commandes artisan utiles

---

1. Vue d'ensemble

Montaza ERP est une application web de gestion d'entreprise développée avec le framework **Laravel 12**. Elle couvre l'ensemble de la chaîne opérationnelle d'une société industrielle : de la prospection commerciale jusqu'à la facturation, en passant par la gestion des stocks, le pilotage des affaires, et le suivi du personnel.

Périmètre fonctionnel

Domaine	Fonctionnalités couvertes
Commercial	Commandes (CDE), Demandes de prix (DDP), Devis Tuyauterie
Projets	Gestion des Affaires, Dossiers Devis, Suivi Tuyauterie
Stock	Matières premières, Mouvements, Alertes stock minimum
Tiers	Sociétés (clients/fournisseurs), Établissements, Contacts
RH	Personnel, Congés, Emploi du temps, Tâches par affaire
Atelier	Parc matériel, Dossiers de réparation
Finance	Facturation des réparations
Admin	Utilisateurs, Rôles (Postes), Permissions, Entités, Logs

---

2. Architecture technique

Stack technologique

```
```

```
┌──────────────────────────────────────────────────────────────┐
│ NAVIGATEUR │
│ Tailwind CSS / Alpine.js / Livewire 3 │
└──────────────────────────┬───────────────────────────────────┘
│ HTTPS
┌──────────────────────────▼───────────────────────────────────┐
│ SERVEUR WEB (Nginx) │
└──────────────────────────┬───────────────────────────────────┘
│
┌──────────────────────────▼───────────────────────────────────┐
│ LARAVEL 12 (PHP 8.3+) │
│ │
│ Routing → Middleware → Controller → Service → Model │
│ │
│ Livewire Components (temps-réel sans rechargement) │
│ Blade Templates (moteur de templates) │
│ Queue / Jobs (traitements asynchrones) │
└──────────────────────────┬───────────────────────────────────┘
│
┌──────────────────────────▼───────────────────────────────────┐
│ PostgreSQL │
└──────────────────────────────────────────────────────────────┘
```

Paquets principaux

Paquet	Rôle
`laravel/framework ^12.0`	Framework PHP
`livewire/livewire ^3.6`	Composants dynamiques PHP/JS
`barryvdh/laravel-dompdf ^3.0`	Génération PDF
`simplesoftwareio/simple-qrcode ^4.2`	Génération QR Code
`laravel-lang/lang ^15.7`	Traductions multilingues
`barryvdh/laravel-debugbar`	Débogage (dev uniquement)
`nunomaduro/phpinsights`	Analyse qualité du code
`phpunit/phpunit ^11.0`	Tests automatisés

---

3. Prérequis et installation

Prérequis système

- PHP **8.3+** avec les extensions : `pdo_pgsql`, `mbstring`, `openssl`, `tokenizer`, `xml`, `ctype`, `json`, `bcmath`, `fileinfo`
- **Composer** (gestionnaire de dépendances PHP)
- **Node.js 18+** et **NPM**
- **PostgreSQL 15+**
- Serveur web **Nginx** ou Apache

Installation

```bash
```

1. Cloner le dépôt

```
git clone <url-du-depot>
cd E6_Montaza_ERP
```

2. Installer les dépendances PHP

composer install

3. Installer les dépendances JavaScript

npm install

4. Configurer l'environnement

cp .env.example .env

php artisan key:generate

5. Configurer la base de données dans .env

DB_CONNECTION=pgsql

DB_HOST=127.0.0.1

DB_PORT=5432

DB_DATABASE=montaza

DB_USERNAME=<user>

DB_PASSWORD=<password>

6. Exécuter les migrations et seeders

```
php artisan migrate:fresh --seed
```

7. Compiler les assets (développement)

npm run dev

8. Compiler les assets (production)

npm run build

9. Lancer le serveur de développement

php artisan serve
```

### Comptes par défaut (après seeding)

| Rôle | Email | Mot de passe |
|---|---|---|
| **Administrateur** | `admin@atlantismontaza.fr` | `Not24get` |
| **Utilisateurs générés** | `<prenom>.<nom>@...` | **format** `InitialeNomAnnée` |

---

## 4. Structure du projet

```
app/
├── Console/Commands/ # Commandes Artisan personnalisées
├── Helpers/
│ ├── NumberHelper.php # Fonctions globales : formatNumber(), formatEuro()
│ └── DateHelper.php # Fonctions globales : formatDate()
├── Http/
│ ├── Controllers/ # Contrôleurs MVC (un par module)
│ ├── Middleware/ # Middlewares personnalisés
│ └── Requests/ # Form Requests (validation)
├── Livewire/ # Composants Livewire (CdeList, DevisTuyauterieForm, MediaSidebar)
├── Mail/ # Mailable classes (emails)
├── Models/ # Modèles Eloquent
├── Observers/ # Observateurs Eloquent
├── Providers/ # Service Providers
├── Services/ # Logique métier (StockService, AccountingService)
└── View/ # View Composers / Components

database/
├── migrations/ # Migrations SQL chronologiques
├── seeders/ # Données initiales et de test
└── factories/ # Factories pour les tests
```
```

```
resources/
├── css/ # Feuilles de style (Tailwind)
├── js/ # JavaScript Alpine.js et utilitaires
└── views/ # Templates Blade par module

routes/
├── web.php # Routes web principale
├── auth.php # Routes d'authentification (Breeze)
├── console.php # Routes de commandes
└── dev_tools.php # Routes outils de développement

tests/
├── Feature/ # Tests fonctionnels (HTTP, Livewire)
└── Unit/ # Tests unitaires (modèles, helpers)
```

---

5. Base de données

Système de gestion : PostgreSQL

L'application utilise PostgreSQL exclusivement. Les requêtes utilisent des clauses spécifiques PostgreSQL comme `ILIKE` (recherche insensible à la casse).

Schéma global (tables principales)

```
┌──────────┐ ┌──────────┐ ┌──────────────┐
│ users │────│ roles │────│ permissions │
└──────────┘ └──────────┘ └──────────────┘
(table pivot role_permission)

┌──────────┐ ┌──────────────┐ ┌──────────────────┐
│ societes │────│etablissements│────│ societe_contacts │
└──────────┘ └──────────────┘ └──────────────────┘
│
│ (societe_matieres)
│
┌──────────┐ ┌──────────┐ ┌──────────┐
│ matieres │────│ stocks │────│mouv_stock │
└──────────┘ └──────────┘ └──────────┘
│ sous_famille_id
```

```
│
┌──────────────┐  ┌──────────┐
│ sous_familles│────│ familles │
└──────────────┘  └──────────┘


┌──────────┐  ┌──────────────┐  ┌──────────┐
│ affaires │────│ cdes │────│cde_ligne│
│ │────│ ddps │────│ddp_ligne│
│ │────│devis_tuyaut. │
│ │────│dossiers_devis│
│ │────│ reparations │
└──────────┘  └──────────────┘
│ (affaire_personnel)
│
┌──────────┐  ┌──────────────────────────┐
│personnel │────│affaire_personnel_taches│
└──────────┘  └──────────────────────────┘
│ (personnel_conges)
│
┌──────────────┐
│pers_conges │
└──────────────┘


┌──────────┐  ┌──────────┐
│ materiels │────│reparation│
└──────────┘  └──────────┘
│
┌────────┐
│factures│
└────────┘


┌──────────────────┐  ┌────────────────────────┐
│devis_tuyauteries│────│devis_tuyauterie_sections│
└──────────────────┘  └────────────────────────┘
│
┌──────────────────────┐
│devis_tuyauterie_lignes│
└──────────────────────┘


┌──────────────────┐  ┌──────────────────────────┐
│ dossiers_devis │────│dossier_devis_quantitatifs│
└──────────────────┘  └──────────────────────────┘
```

Chronologie des migrations clés

Date	Migration	Objet
2024-10-28	`create_model_changes_table`	Audit/historique
2024-10-30	`create_societes_table`	Tiers
2024-12-16	`create_table_matiere`	Catalogue matières
2025-01-08	`create_ddps_table`, `create_cdes_table`	Commercial
2025-02-13	`add_table_mouvement`	Mouvements de stock
2025-05-22	`create_media_table`	Gestion médias
2025-06-24	`create_affaires_table`	Affaires/Projets
2025-10-24	`materiel`, `reparation`	Atelier/SAV
2025-11-18	`create_table_facture`	Facturation
2026-01-19	`create_devis_tuyauterie_tables`	Devis tuyauterie
2026-02-11	`create_personnels_table`, `affaire_personnel`, `taches`	RH
2026-02-12	`create_dossiers_devis_tables`	Dossiers devis
2026-03-09	`create_affaire_suivi_lignes_table`	Suivi affaires
2026-03-26	`add_table_charge_affaire_devis`	Charges affaire/devis

---

6. Modules fonctionnels

6.1 Gestion commerciale — Commandes (CDE)

Les **Commandes** (`cdes`) représentent les bons de commande émis vers les fournisseurs.

Cycle de vie d'une CDE

```
[Brouillon] → [En cours] → [Terminée / Livrée] → [Vérifiée]
↓
```

[Annulée]
```

Les statuts sont gérés via la table `ddp_cde_statuts`. Le statut `id=3` (Terminée) déclenche automatiquement l'entrée en stock des lignes livrées.

### *Champs clés de la table `cdes`*

| Champ | Type | Description |
|---|---|---|
| `code` | string | Identifiant lisible (ex: CDE-2026-001) |
| `nom` | string | Désignation libre |
| `ddp_cde_statut_id` | FK | Statut du document |
| `affaire_id` | FK nullable | Lien vers une affaire |
| `ddp_id` | FK nullable | Lien vers la DDP d'origine |
| `entite_id` | FK | Entité émettrice |
| `user_id` | FK | Créateur |
| `total_ht` | decimal | Montant HT calculé |
| `tva` | decimal | TVA |
| `total_ttc` | decimal | Montant TTC calculé |
| `frais_de_port` | decimal | Frais de port |
| `adresse_livraison` | text | Adresse de livraison |
| `adresse_facturation` | text | Adresse de facturation |
| `condition_paiement_id` | FK | Condition de paiement |

### *Lignes de commande (`cde_lignes`)*

Chaque CDE contient N lignes triées par `poste` :

| Champ | Type | Description |
|---|---|---|
| `poste` | integer | Numéro de ligne |
```

`matiere_id`	FK nullable	Matière commandée
`designation`	string	Désignation libre
`quantite`	decimal	Quantité
`prix_unitaire`	decimal	Prix unitaire HT
`date_livraison`	date	Date de livraison prévue
`date_livraison_reelle`	date	Date réelle
`is_stocke`	boolean	Indique si l'entrée en stock a été faite
`non_livre`	boolean	Marque une ligne comme non livrée

Logique métier automatique

Lorsqu'une CDE passe au statut **Terminée** (`id=3`), l'événement `updated` du modèle `Cde` itère sur toutes les lignes qui :

- Ne sont pas encore stockées (`is_stocke = false`)
- Ont une `matiere_id` définie
- Ont une `date_livraison_reelle` renseignée

Et appelle `StockService->stock()` pour créer l'entrée en stock correspondante.

De plus, à chaque sauvegarde ou suppression d'une CDE liée à une affaire, `$affaire->updateTotal()` est appelé pour recalculer le budget engagé.

6.2 Demandes de Prix (DDP)

Les **DDP** (`ddps`) sont les demandes de prix envoyées aux fournisseurs en amont d'une commande.

Modèle de données

```
ddps
└── ddp_lignes (articles demandés)
└── ddp_ligne_fournisseurs (réponses par fournisseur/contact)
```

Chaque ligne de DDP peut recevoir plusieurs propositions de différents fournisseurs, permettant la comparaison de prix avant de créer une CDE.

6.3 Affaires

Les **Affaires** (`affaires`) représentent les projets ou chantiers de l'entreprise. Elles constituent le pivot central de l'application, agrégant l'ensemble des documents et ressources liés à un projet.

Statuts

Constante	Valeur DB	Label	Couleur
`STATUT_EN_ATTENTE`	`en_attente`	En attente	Gris
`STATUT_EN_COURS`	`en_cours`	En cours	Bleu
`STATUT_TERMINE`	`termine`	Terminé	Vert
`STATUT_ARCHIVE`	`archive`	Archivé	Rouge

Relations d'une Affaire

```php
Affaire
├── hasMany(Cde) // Commandes
├── hasMany(Ddp) // Demandes de prix
├── hasMany(Reparation) // Réparations
├── hasMany(DevisTuyauterie) // Devis tuyauterie
├── hasMany(DossierDevis) // Dossiers devis
├── hasMany(AffaireSuiviLigne) // Lignes de suivi
├── belongsToMany(Materiel) // Parc matériel (pivot: affaire_materiel)
└── belongsToMany(Personnel) // Équipe (pivot: affaire_personnel)
└── hasMany(AffairePersonnelTache) // Tâches
```

Calcul automatique du total HT

La méthode `updateTotal()` calcule automatiquement le `total_ht` de l'affaire en agrégeant :

- Les **CDE** aux statuts En cours (2), Terminée (3) et Vérifiée (5)
- Les **Devis Tuyauterie** (non archivés)
- Les **Factures** des réparations

Assignation du personnel et tâches

Chaque personnel peut être assigné à plusieurs affaires via la table pivot `affaire_personnel` (modèle `AffairePersonnel`, pivot personnalisé). Pour chaque assignation, des **tâches** peuvent être créées (`affaire_personnel_taches`) avec :

- Titre, description
- Date de début / fin
- Statut : `a_faire`, `en_cours`, `termine`
- Priorité : `basse`, `normale`, `haute`

Suivi des lignes tuyauterie

Les `AffaireSuiviLigne` permettent de documenter l'avancement d'un chantier de tuyauterie associé à l'affaire.

6.4 Devis Tuyauterie

Le module **Devis Tuyauterie** (`devis_tuyauteries`) gère la création de devis structurés pour les chantiers de tuyauterie/chaudronnerie.

Structure

```
devis_tuyauteries
└── devis_tuyauterie_sections (sections logiques du devis)
└── devis_tuyauterie_lignes (lignes détaillées, avec matiere_id optionnel)

devis_tuyauteries
└── devis_stock_reservations (réservations de stock matière)
└── charge_affaire_devis (Users responsables du devis N-N)
```

Champs notables

- `reference_projet` / `lieu_intervention` : informations projet
- `is_archived` : archivage logique (boolean)
- `options` : JSON — options configurables du devis
- `total_ht`, `total_tva`, `total_ttc`, `marge_globale` : totaux calculés
- `affaire_id` : lien optionnel vers une affaire
- `dossier_devis_id` : lien optionnel vers un dossier devis

6.5 Dossiers Devis

Les **Dossiers Devis** (`dossiers_devis`) regroupent les quantitatifs et devis pour un projet.

Statuts

Constante	Valeur	Description
`STATUT_QUANTITATIF`	`quantitatif`	Phase métrée
`STATUT_EN_DEVIS`	`en_devis`	Devis en cours
`STATUT_VALIDE`	`valide`	Devis validé
`STATUT_ARCHIVE`	`archive`	Archivé

Numérotation automatique

Format : `DD-YYYY-NNN` (ex: `DD-2026-001`). La méthode statique `generateNextCode()` incrémente automatiquement le dernier numéro de l'année en cours.

6.6 Stock & Matières

Catalogue de matières (`matieres`)

Chaque matière possède une arborescence : **Famille → Sous-famille → Matière**.

Champs clés :

Champ	Description
`ref_interne`	Référence interne unique
`designation`	Libellé de l'article
`unite_id`	Unité de mesure
`sous_famille_id`	Classification
`dn`	Diamètre nominal (pour tuyauterie)
`epaisseur`	Épaisseur (pour tubes/plaques)

`standard_version_id`	**Norme applicable**
`stock_min`	**Seuil d'alerte stock**
`material_id`	**Matériau (acier, inox, etc.)**

À la **création** d'une matière, un enregistrement `stocks` est automatiquement créé avec `quantite=0`.

Gestion des stocks (`stocks`)

La table `stocks` stocke la quantité par matière **et** par valeur unitaire (permettant le suivi multi-lots) :

```
stocks
├── matiere_id (FK)
├── quantite
├── valeur_unitaire
└── certificat (numéro de certificat matière)
```

Mouvements de stock (`mouvement_stocks`)

Chaque entrée/sortie est tracée avec : `matiere_id`, `type` (entrée/sortie), `quantite`, `valeur_unitaire`, `raison`, `cde_ligne_id`.

Service StockService

Le `StockService` centralise toute la logique de mouvement de stock :

```php
$stockService->stock(
matiereId: $ligne->matiere_id,
type: 'entree', // ou 'sortie'
quantite: 10.5,
valeurUnitaire: 25.00,
raison: 'Livraison CDE-2026-001',
cde_ligne_id: $ligne->id
);
```

Le service gère deux modes d'affichage du stock (`typeAffichageStock()`) :

- **Type 1** : stock unique par matière

- **Type 2** : stock multi-lots par valeur unitaire

Alertes stock minimum

À chaque mouvement de sortie, le service vérifie si le stock passe sous le `stock_min`. Si c'est le cas et que la notification n'a pas déjà été envoyée (`stock_min_notif_envoyee = false`), une notification est créée et le flag est levé. Quand le stock repasse au-dessus du seuil, le flag est réinitialisé.

Prix fournisseurs (`societe_matiere_prix`)

Chaque association entre une société et une matière (`societe_matieres`) peut contenir :

- `ref_fournisseur` : référence chez ce fournisseur
- `designation_fournisseur` : libellé fournisseur
- `prix` : dernier prix connu
- `date_dernier_prix` : date du prix

6.7 Tiers — Sociétés & Contacts

Sociétés (`societes`)

Les sociétés peuvent être de plusieurs natures (géré par `societe_type_id`) :

Type ID	Nature
1	Client
2	Fournisseur
3	Client & Fournisseur

Des **scopes Eloquent** permettent de filtrer : `scopeClients()` et `scopeFournisseurs()`.

Hiérarchie

```
Societe (siège social)
└── Etablissement (site/agence)
└── SocieteContact (interlocuteur)
```

Les contacts qui appartiennent directement à un établissement sont accessibles via `hasManyThrough` depuis la

Societe.

6.8 Personnel & Congés

Modèle Personnel

Le modèle `Personnel` utilise `SoftDeletes` pour gérer les anciens employés (qui restent visibles via un filtre dédié dans l'interface).

Champs principaux :

Champ	Description
`matricule`	Numéro d'employé
`statut`	Actif / Inactif
`date_embauche` / `date_depart`	Dates d'activité
`raison_depart` / `motif_depart`	Gestion du départ
`poste` / `departement`	Organisation

Congés (`personnel_conges`)

Les congés sont créés avec un statut (`en_attente`, `valide`, `refuse`) et une période de validité (`date_debut`, `date_fin`).

La méthode `estEnConge($date)` retourne un boolean testant si l'employé est en congé validé à une date donnée.

Emploi du temps

L'interface emploi du temps (`PersonnelEmploiDuTempsController`) présente une vision chronologique de toutes les affaires et tâches d'un employé.

6.9 Atelier / SAV — Matériel & Réparations

Parc matériel (`materiels`)

Champ	Description

`reference`	Référence unique du matériel
`designation`	Nom
`numero_serie`	Numéro de série
`status`	`actif`, `maintenance`, `inactif`
`desactive`	Boolean — force le statut à `inactif`
`acquisition_date`	Date d'acquisition

Un matériel peut être assigné à une affaire (table pivot `affaire_materiel` avec `date_debut`, `date_fin`, `statut`). Quand une affaire passe en `termine` ou `archive`, toutes les assignations actives sont automatiquement clôturées (date_fin = now()).

Réparations (`reparations`)

Une réparation relie un `Materiel` à un `User` (responsable) et optionnellement une `Affaire`. Elle peut avoir plusieurs `Facture`.

6.10 Facturation

Les **Factures** (`factures`) sont attachées aux réparations et utilisées pour calculer le coût réel d'un chantier dans le récapitulatif de l'affaire.

Champs : `numero_facture`, `date_emission`, `montant_total`, `reparation_id`.

À chaque sauvegarde ou suppression d'une facture, `affaire->updateTotal()` est déclenché.

6.11 Administration

Entités (`entites`)

Les entités représentent les sociétés légales qui émettent les documents (CDE, DDP). Elles sont partagées sur toutes les vues via le middleware `GetGlobalVariable`.

Paramètres globaux (`app_settings`)

Le contrôleur `AppSettingController` (protégé par la permission `gerer_les_utilisateurs`) gère les paramètres

globaux de l'application.

Données de référence

Le contrôleur `ReferenceDataController` (protégé par `gerer_les_donnees_de_reference`) gère via une interface CRUD unifiée :

- Familles / Sous-familles
- Formes juridiques
- Dossiers standards
- Pays
- Codes APE
- Conditions de paiement
- Matériaux (`materials`)
- Unités

---

7. Couche modèles (Eloquent)

Liste des modèles

Modèle	Table	Description
`User`	`users`	Utilisateurs de l'application
`Role`	`roles`	Postes / rôles
`Permission`	`permissions`	Permissions RBAC
`Entite`	`entites`	Entités légales émettrices
`Societe`	`societes`	Tiers (clients/fournisseurs)
`Etablissement`	`etablissements`	Sites d'une société
`SocieteContact`	`societe_contacts`	Contacts
`Matiere`	`matieres`	Catalogue matières
`Stock`	`stocks`	État du stock par matière+lot
`MouvementStock`	`mouvement_stocks`	Historique mouvements
`Cde`	`cdes`	Commandes

CdeLigne	cde_lignes	Lignes de commande
CdeNote	cde_notes	Notes sur commandees
Ddp	ddps	Demandes de prix
DdpLigne	ddp_lignes	Lignes DDP
DdpLigneFournisseur	ddp_ligne_fournisseurs	Réponses fournisseurs
Affaire	affaires	Projets/Chantiers
AffairePersonnel	affaire_personnel	Pivot Affaire↔Personnel
AffairePersonnelTache	affaire_personnel_taches	Tâches du personnel
AffaireSuiviLigne	affaire_suivi_lignes	Suivi tuyauterie
DevisTuyauterie	devis_tuyauteries	Devis tuyauterie
DevisTuyauterieSection	devis_tuyauterie_sections	Sections du devis
DevisTuyauterieLigne	devis_tuyauterie_lignes	Lignes du devis
DevisStockReservation	devis_stock_reservations	Réservations stock
DossierDevis	dossiers_devis	Dossiers de devis
DossierDevisQuantitatif	dossier_devis_quantitatifs	Quantitatifs
Personnel	personnels	Employés
PersonnelConge	personnel_conges	Congés
Materiel	materiels	Parc matériel
Reparation	reparations	Dossiers réparation
Facture	factures	Factures
Notification	notifications	Notifications internes
ModelChange	model_changes	Audit trail
Media	media	Fichiers attachés
MediaType	media_types	Types de fichiers

`Mailtemplate`	`mailtemplates`	**Modèles d'email**
`Famille`	`familles`	**Familles de matières**
`SousFamille`	`sous_familles`	**Sous-familles**
`Commentaire`	`commentaires`	**Commentaires libres**
`ConditionPaiement`	`condition_paiements`	**Conditions paiement**
`TypeExpedition`	`type_expeditions`	**Modes d'expédition**
`Unite`	`unites`	**Unités de mesure**
`Standard`	`standards`	**Normes**
`StandardVersion`	`standard_versions`	**Versions de normes**
`Material`	`materials`	**Matériaux (acier, inox...)**
`Pays`	`pays`	**Pays**
`FormeJuridique`	`forme_juridiques`	**Formes juridiques**
`CodeApe`	`code_apes`	**Codes APE/NAF**
`SocieteType`	`societe_types`	**Types de société**
`AppSetting`	`app_settings`	**Paramètres globaux**
`PredefinedShortcut`	`predefined_shortcuts`	**Raccourcis prédéfinis**
`UserShortcut`	`user_shortcuts`	**Raccourcis utilisateur**

Trait MediaableTrait

Le trait `MediaableTrait` est utilisé par `Cde` et `Ddp` pour leur associer des fichiers (médias). Il fournit les relations et méthodes nécessaires à la gestion des pièces jointes.

Événements Eloquent & Audit

Les modèles `User`, `Role`, `Permission`, `Societe`, `Matiere` déclenchent automatiquement l'enregistrement d'un `ModelChange` lors de chaque création, modification ou suppression (via des callbacks `booted()`).

---

8. Contrôleurs

Convention

Tous les contrôleurs héritent de `App\Http\Controllers\Controller`. Chaque méthode retourne explicitement un type (`View`, `JsonResponse`, `RedirectResponse`).

Catalogue des contrôleurs

Contrôleur	Périmètre
`AdministrationController`	Entités, infos entreprise
`AppSettingController`	Paramètres globaux
`AffaireController`	CRUD Affaires, personnel, tâches
`CdeController`	CRUD Commandes, PDF, emails
`CdeNoteController`	Notes sur commandes
`DdpController`	CRUD Demandes de prix
`DevisTuyauterieController`	CRUD Devis tuyauterie
`DossierDevisController`	CRUD Dossiers devis
`EntiteController`	Entités légales
`EtablissementController`	Établissements
`FactureController`	Factures réparations
`FormeJuridiqueController`	Formes juridiques
`MailtemplateController`	Modèles emails
`MaterielController`	Parc matériel
`MatiereController`	Catalogue matières
`MatierePrixController`	Prix fournisseurs
`MediaController`	Gestion fichiers
`ModelChangeController`	Audit trail

`NotificationController`	Notifications internes
`PaysController`	Référentiel pays
`PermissionController`	RBAC permissions
`PersonnelController`	CRUD Personnel, congés
`PersonnelEmploiDuTempsController`	Emploi du temps
`ProductionController`	Module production
`ProfileController`	Profil utilisateur
`ReferenceDataController`	Données référentielles unifiées
`ReparationController`	Dossiers réparation
`RoleController`	Postes / rôles
`SocieteController`	CRUD Sociétés
`SocieteContactController`	Contacts
`StandardController`	Normes
`StockController`	Stock, mouvements
`UserShortcutController`	Raccourcis utilisateur
`DocumentationController`	Documentation intégrée
`CodeApeController`	Codes APE
`CommentaireController`	Commentaires

---

9. Services métier

StockService (`app/Services/StockService.php`)

Centralise la gestion des entrées/sorties de stock.

Méthode principale :

```php
```

```
public function stock(
int $matiereId,
string $type, // 'entree' | 'sortie'
float $quantite,
?float $valeurUnitaire = null,
?string $raison = null,
$cde_ligne_id = null
): array
```

Comportement :

1. Récupère la matière (`findOrFail`)
2. Selon le type d'affichage de stock (1 ou 2), gère la mise à jour des quantités dans `stocks`
3. Crée un `MouvementStock` pour la traçabilité
4. Appelle `checkStockMinimum()` pour vérifier les alertes

Injection de dépendances :

Le service est injecté dans le constructeur de `CdeController` via le conteneur IoC de Laravel.

AccountingService (`app/Services/AccountingService.php`)

Fichier présent mais non implémenté. Prévu pour la gestion comptable avancée.

---

10. Middleware

Middleware	Alias	Rôle
`GetGlobalVariable`	`GetGlobalVariable`	Partage notifications, entités, raccourcis sur toutes les vues
`XSSProtection`	`XSSProtection`	Nettoyage des inputs (caractères `<`, `>`, `;`, `--`)
`CheckPermission`	`permission`	Vérifie qu'une permission nommée est accordée à l'utilisateur
`CheckRole`	_(non configuré)_	Vérification de rôle
`PreventDebugMode`	_(non configuré)_	Bloque l'accès en mode debug en production
`VerifyCsrfToken`	_(Laravel)_	Protection CSRF sur les requêtes POST/PATCH/DELETE

GetGlobalVariable

Ce middleware s'exécute sur **toutes les routes** (authentifiées ou non) et partage via `View::share()` :

- `_notifications_count` : nombre de notifications non lues
- `_notificationsSystem_count` : notifications système non lues
- `_notificationsStock_count` : alertes stock non lues
- `_entites` : liste de toutes les entités
- `_shortcuts` : raccourcis de l'utilisateur courant
- `societeTypes` : types de sociétés

XSSProtection

S'applique à tous les formulaires. Remplace les caractères dangereux par la chaîne `CARACTERE_NON_AUTORISE` (sauf les apostrophes correctement échappées) dans tous les champs de la requête de manière récursive.

CheckPermission

Usage dans les routes :
```php
Route::middleware('permission:gerer_les_utilisateurs')->group(function () { ... });
```

---

11. Composants Livewire

Livewire v3 est utilisé pour les interfaces interactives sans rechargement de page.

CdeList (`app/Livewire/CdeList.php`)

Composant de liste des commandes avec filtrage dynamique.

DevisTuyauterieForm (`app/Livewire/DevisTuyauterieForm.php`)

Formulaire interactif pour la création et modification des devis tuyauterie. Gère les sections et lignes de manière dynamique (ajout/suppression sans rechargement de page).

MediaSidebar (`app/Livewire/MediaSidebar.php`)

Panneau latéral contextuel pour afficher, uploader et gérer les médias attachés à un document (CDE, DDP...). Utilise le trait `MediaableTrait`.

---

12. Helpers globaux

Définis dans `app/Helpers/` et auto-chargés via `composer.json`.

NumberHelper (`app/Helpers/NumberHelper.php`)

```php
// Formate un nombre avec séparateur de milliers
formatNumber(1234567.89) // → "1 234 567.89"
formatNumber(1234567.89, true) // → "1234567.89" (sans espace)

// Formate en euros
formatEuro(1234.5) // → "1 234,50 €"
```

DateHelper (`app/Helpers/DateHelper.php`)

```php
// Formate une date en français
formatDate('2026-04-20') // → "20/04/2026"
formatDate('2026-04-20 14:30:00', avec_heure: true) // → "20/04/2026 14:30"
```

---

13. Système de notifications

Les notifications internes (`notifications`) sont ciblées par **rôle** (pas par utilisateur individuel).

Création d'une notification

```php
Notification::createNotification(
role: $role, // Rôle destinataire
type: 'stock', // 'stock'
```

'system'

```
...
title: 'Stock minimum atteint',
message: 'Le stock de X est bas.',
action_requise: 'Voir le stock',
route_nom: 'matieres.show',
route_data: ['matiere' => $id],
label: 'Voir le stock'
```

);
```

### Affichage

Le middleware `GetGlobalVariable` transmet le compteur `_notifications_count` à toutes les vues, utilisé pour le badge dans la barre de navigation.

Les routes dédiées permettent de lire ( `lu` ), marquer comme non-lu ( `nonLu` ), et transférer les notifications.

---

## 14. Système de permissions & rôles

### Architecture RBAC

```
User → Role → Permission(s)
```

- Un utilisateur a **exactement un rôle** (`role_id`)
- Un rôle a **plusieurs permissions** (`role_permission` — table pivot)
- Les permissions sont **nommées** (ex: `gerer_les_utilisateurs` , `voir_les_societes`)

Permissions connues

Permission	Description
`gerer_les_utilisateurs`	Accès à l'administration des utilisateurs et personnel
`gerer_les_permissions`	Modification de la matrice de permissions
`gerer_les_postes`	Gestion des rôles
`gerer_les_donnees_de_reference`	CRUD données référentielles
`voir_les_societes`	Accès au module sociétés
`voir_historique`	Accès aux logs de modifications

Vérification dans le code

```php
// Dans un middleware de route

```
Route::middleware('permission:gerer_les_utilisateurs')->group(...);

// Dans un contrôleur / vue
if ($user->permissions()->contains('name', 'voir_les_societes')) { ... }
```

---

15. Gestion des médias

Le système de médias permet d'attacher des fichiers à des documents (CDE, DDP).

Trait MediaableTrait

Les modèles `Cde` et `Ddp` utilisent `MediaableTrait` pour obtenir :

- La relation `media()`
- Les méthodes d'upload, de liste et de suppression

Composant Livewire MediaSidebar

Le sidebar s'ouvre contextuellement sur les pages de détail d'un document. Il permet :

- L'upload de nouveaux fichiers
- La consultation par type (`media_types`)
- La suppression

---

16. Historique des modifications (Audit)

La table `model_changes` enregistre chaque création, modification et suppression des modèles sensibles.

Structure

Champ	Description
`user_id`	Utilisateur ayant effectué l'action
`model_type`	Type de modèle (ex: `matiere`, `Postes`)
`event`	`creating`, `updating`, `deleting`
`before`	JSON de l'état avant modification

`after`	JSON de l'état après modification

Modèles audités

`User`, `Role`, `Permission`, `Societe`, `Matiere`

Accès

La liste des logs est accessible via `GET /logs` (permission `voir_historique`) dans `ModelChangeController`.

---

17. Routage

Fichiers de routes

Fichier	Périmètre
`routes/web.php`	Routes principales de l'application
`routes/auth.php`	Authentification (Laravel Breeze)
`routes/console.php`	Commandes console planifiées
`routes/dev_tools.php`	Outils de développement

Middlewares globaux de groupe

Toutes les routes protégées sont dans un groupe :

```php
Route::middleware(['GetGlobalVariable', 'XSSProtection', 'auth'])->group(function () { ... });
```

Conventions de nommage des routes

Pattern	Exemple
`{module}.index`	`cdes.index`
`{module}.show`	`affaires.show`
`{module}.create`	`affaires.create`

`{module}.store`	`affaires.store`
`{module}.edit`	`affaires.edit`
`{module}.update`	`affaires.update`
`{module}.destroy`	`affaires.destroy`
`{module}.restore`	`affaires.restore`

Routes notables

```
GET / → Page d'accueil (accueil ou welcome)
GET /dashboard → Tableau de bord
GET /affaires → Liste des affaires
GET /affaire/{affaire} → Détail affaire
GET /cdes → Liste des commandes
GET /ddps → Liste des DDP
GET /societes → Liste des sociétés
GET /matieres → Catalogue matières
GET /stock → Gestion du stock
GET /personnel → Liste du personnel
POST /personnel/{personnel}/conges → Créer un congé
GET /personnel/{personnel}/emploi-du-temps → Emploi du temps
GET /reparations → Liste des réparations
GET /devis-tuyauterie → Liste des devis
GET /dossiers-devis → Liste des dossiers devis
GET /administration → Tableau de bord admin
GET /logs → Historique des modifications
GET /notifications → Centre de notifications
GET /documentation → Documentation intégrée
```

---

18. Frontend

Stack

Technologie	Usage

Blade	**Moteur de templates PHP (layouts, composants, vues)**
Livewire 3	**Composants PHP dynamiques (liste CDE, formulaire devis)**
Tailwind CSS	**Framework CSS utilitaire**
Alpine.js	**JavaScript léger pour interactions UI (menus, modales)**
Vite	**Bundler (remplace Webpack)**

Organisation des vues

```
resources/views/
├── layouts/ # Layout principal (nav, footer, sidebar)
├── components/ # Composants Blade réutilisables
├── accueil.blade.php # Page d'accueil
├── dashboard.blade.php
├── affaires/ # Vues du module Affaires
├── ddp_cde/ # Vues CDE et DDP
├── devis_tuyauterie/ # Vues devis tuyauterie
├── dossiers_devis/ # Vues dossiers devis
├── matieres/ # Catalogue matières
├── societes/ # Tiers
├── personnel/ # RH
├── production/ # Module production
├── reparation/ # SAV
├── administration/ # Admin
├── notifications/ # Centre de notifications
├── media/ # Gestion médias
├── emails/ # Templates emails
└── documentation/ # Documentation intégrée
```

Génération PDF

L'application utilise **DomPDF** (`barryvdh/laravel-dompdf`) pour la génération des bons de commande et documents en PDF.

QR Codes

Le package `simplesoftwareio/simple-qrcode` est utilisé pour générer des QR codes sur les documents

imprimables.

---

19. Sécurité

Authentification

Gérée par **Laravel Breeze** (routes dans `routes/auth.php`). Fonctionnalités :

- Connexion / déconnexion
- Réinitialisation de mot de passe
- Vérification d'email

Autorisation (RBAC)

Le middleware `CheckPermission` aborte avec un code HTTP `403` si l'utilisateur n'a pas la permission requise. Aucune route sensible n'est accessible sans la vérification.

Protection CSRF

Toutes les requêtes `POST`, `PATCH`, `PUT`, `DELETE` sont protégées par le middleware `VerifyCsrfToken` de Laravel.

Protection XSS

Le middleware `XSSProtection` s'applique à l'ensemble des routes pour nettoyer les inputs utilisateur avant traitement.

Protection des données sensibles

- Les mots de passe sont hashés par Bcrypt (standard Laravel `Hash::make()`)
- Le champ `password` est exclu de la sérialisation JSON (`$hidden` dans `User`)
- Le token `remember_token` est ignoré dans l'audit trail

Sécurité base de données

- Utilisation exclusive de l'**ORM Eloquent** (protection injection SQL via liaisons préparées)
- Les inputs validés via des **Form Requests** avant toute persistance
- Validation systématique des types, longueurs et existence FK (`exists:table,id`)

---

20. Tests

Organisation

```
tests/
├── Feature/
│ ├── Auth/ # Tests d'authentification
│ ├── Livewire/ # Tests composants Livewire
│ ├── DevisTuyauterieControllerTest.php
│ ├── PersonnelControllerTest.php
│ ├── ProfileTest.php
│ ├── ReparationControllerTest.php
│ └── ReparationTest.php
└── Unit/
├── AffaireTest.php # Tests modèle Affaire
├── HelpersTest.php # Tests helpers (formatNumber, formatDate)
├── ExampleTest.php
└── Models/ # Tests unitaires modèles
```

Lancer les tests

```bash
```

Tous les tests

php artisan test

Tests unitaires seulement

```
php artisan test --testsuite=Unit
```

Tests fonctionnels seulement

```
php artisan test --testsuite=Feature
```

Fichier spécifique

php artisan test tests/Feature/PersonnelControllerTest.php

Avec couverture de code

php artisan test --coverage
```

### Commandes batch disponibles

Des scripts `.bat` sont fournis pour lancer des suites de tests spécifiques :

- `TEST_ALL.bat` — tous les tests
- `TEST_CDE.bat` — tests commandes
- `TEST_CDELIGNES.bat` — tests lignes de commande
- `TEST_PERSONNEL.bat` — tests personnel

### Configuration

Le fichier `phpunit.xml` définit les suites de tests et la connexion à la base de données de test (SQLite in-memory ou PostgreSQL de test selon l'environnement).

---

## 21. Déploiement

### Configuration Nginx

Un fichier `nginx.conf` est fourni à la racine du projet. Configuration typique pour Laravel :

```nginx
root /var/www/html/public;
index index.php;

location / {
try_files $uri $uri/ /index.php?$query_string;
}

location ~ \.php$ {
fastcgi_pass unix:/var/run/php/php8.3-fpm.sock;
include fastcgi_params;
fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
}
```
```

Variables d'environnement critiques

```env
APP_ENV=production
APP_DEBUG=false
APP_URL=https://votre-domaine.fr

DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=montaza
DB_USERNAME=<user>
DB_PASSWORD=<password>

MAIL_MAILER=smtp
MAIL_HOST=<smtp-host>
MAIL_PORT=587
MAIL_FROM_ADDRESS=noreply@votre-domaine.fr

QUEUE_CONNECTION=database
```

Optimisations pour la production

```bash
```

Cacher la configuration

php artisan config:cache

Cacher les routes

php artisan route:cache

Cacher les vues

php artisan view:cache

Optimiser l'autoloader Composer

composer install --optimize-autoloader --no-dev

Compiler les assets

npm run build
```
```

Permissions de stockage

```bash
chmod -R 775 storage bootstrap/cache
chown -R www-data:www-data storage bootstrap/cache
```

Un script `fix-storage-permissions.sh` est fourni pour configurer ces permissions automatiquement.

---

22. Commandes artisan utiles

```bash
```

Rafraîchir la BDD (DÉTRUIT TOUTES LES DONNÉES)

```
php artisan migrate:fresh --seed
```

Exécuter uniquement les migrations en attente

php artisan migrate

Rollback de la dernière migration

```
php artisan migrate:rollback
```

Générer la clé d'application

```
php artisan key:generate
```

Générer les fichiers d'aide IDE

```
php artisan ide-helper:generate
```

```
php artisan ide-helper:models
```

Vider les caches

```
php artisan cache:clear
php artisan config:clear
php artisan route:clear
php artisan view:clear
```

Lancer la console interactive

```
php artisan tinker
```

Lancer les tests avec rapport de couverture

```
php artisan test --coverage
```

Analyser la qualité du code (PHPInsights)

php artisan insights
```

---

*Documentation générée le 20/04/2026 — Montaza ERP v1.x (Laravel 12)*
```